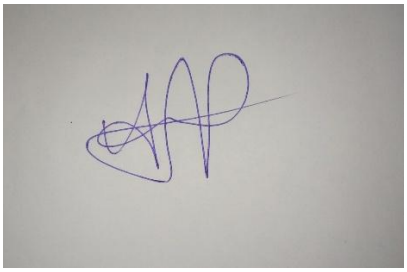
Name:	**Mohammed Arshad R**	**Reg no:**	**192521126**
Subject:	**Cryptography and Network Security**		
Assignment:	**Annexure A - Selected Tasks (Question 1 & Question 2)**		

Code of Conduct:

I Mohammed Arshad R/192521126 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



CRYPTOGRAPHY & NETWORK SECURITY ASSIGNMENT

QUESTION 1: SECURE PASSWORD STORAGE IN WEB APPLICATIONS

---

1. Problem Statement & Executive Context

A web application stores user passwords and needs to protect them against database breaches. Web applications are constantly exposed to adversarial threats, ranging from SQL Injection (SQLi) vulnerabilities and server-side remote code execution to physical database storage compromises and unauthorized cloud snapshot access. When an attacker successfully breaches the underlying database layer, the primary defense preserving user account integrity is the security mechanism applied to stored credentials.

2. Part A: Vulnerabilities and Risks of Plaintext Password Storage

Storing passwords in unencrypted, plaintext format represents one of the most critical security flaws in software architecture. The underlying risks and technical implications include:

2.1. Direct Exposure upon Data Leakage

If an attacker gains read-only access to the database via SQL injection or compromised backups, every account credential is immediately exposed without requiring any decryption effort. The attacker can extract the entire user directory and log in as any user, including system administrators.

2.2. The Threat of Credential Stuffing Attacks

Users frequently reuse identical or slightly modified passwords across multiple online platforms (e.g., email, banking, social media, and enterprise portals). A plaintext breach on one web application compromises the user's security footprint across the entire Internet, facilitating automated credential stuffing attacks against external services.

2.3. Insider Threats and System Administrator Abuse

Plaintext storage grants privileged users (database administrators, system engineers, and developers) direct access to sensitive credentials. This creates insider threats, compliance violations (e.g., GDPR, PCIDSS, HIPAA), and loss of user trust.

2.4. Limitations of Reversible Encryption

Attempting to protect credentials using reversible symmetric (e.g., AES-GCM) or asymmetric (e.g., RSA) encryption is fundamentally flawed for password storage. If an attacker compromises the application server, they can retrieve the decryption key stored in configuration files or memory, rendering the encryption useless.

3. Part B: Recommended Secure Storage Architecture

To protect user credentials against database compromises, web applications must employ **cryptographic password hashing with unique salts**, using dynamic key-derivation algorithms such as **Argon2id** or **Bcrypt**.

3.1. Cryptographic One-Way Hashing

Unlike encryption, cryptographic hash functions are strictly one-way mathematical algorithms $H: \{0,1\}^* > \{0,1\}^n$. Given a digest $h = H(P)$, it is computationally infeasible to reconstruct the original plaintext password P (Pre-Image Resistance).

3.2. Salt Injection & Mitigation of Pre-Computed Attacks

Naive hashing (e.g., SHA-256 without a salt) is vulnerable to **Rainbow Table Attacks** and reverse lookup dictionaries, where attackers pre-compute hashes for billions of common passwords. Furthermore, identical passwords produce identical hashes, revealing shared credentials across users.

A **Salt** is a cryptographically secure random sequence of bytes (minimum 128 bits) generated independently for each user upon account creation or password modification. The salt is concatenated with the password before hashing:

$$Stored\ Hash = KeyDerivationFunction(Password, Salt, CostParameters)$$ The

salt is stored alongside the hash in the database in plaintext. Salt injection guarantees that:

- Identical passwords produce completely unique hash digests for different accounts.
- Pre-computed rainbow tables are rendered obsolete, forcing attackers to compute hashes individually per target account.

3.3. Adaptive Work Factors (Memory-Hard & CPU-Bound Functions)

Standard general-purpose cryptographic hash functions like SHA-256 or SHA-3 are engineered for high throughput (processing gigabytes per second). Attackers leveraging specialized hardware (GPUs, ASICs, FPGAs) can execute billions of SHA-256 guesses per second.

Secure password storage requires **Adaptive Key Derivation Functions (KDFs)** that introduce deliberate computational and memory overheads:

- **Argon2id (Winner of the Password Hashing Competition):** Combines Argon2i (resistant to sidechannel attacks) and Argon2d (resistant to GPU cracking). It enforces configurable memory usage, time iterations, and parallelism, constraining an attacker's ability to run parallel hardware cracking jobs.
- **Bcrypt:** Based on the Blowfish cipher, Bcrypt incorporates an adaptive cost factor 2^k, allowing administrators to scale hashing latency as hardware performance increases over time.

4. Mathematical & Architectural Workflow

The credential storage and authentication flow operates as follows:

Phase	Operation Steps	Cryptographic Guarantee
Registration	1. Generate random 128-bit Salt S. 2. Compute H = Argon2id(Password, Salt,m, t, p). 3. Store (User, Salt, H) in Database.	Plaintext password is immediately wiped from memory.
Authentication	1. Fetch stored salt S and hash H for targetuser. 2. Compute H' = Argon2id(Password_input, Salt, m, t, p). 3. Perform constant-time comparison H'== H.	Verifies identity without revealing or reconstructing P. Prevents timing attacks.

5. Python Reference Implementation: Secure Password Management

The following Python script demonstrates secure password hashing and verification using the standard `hashlib` PBKDF2 implementation with HMAC-SHA256 and unique 128-bit salts.

```
import hashlib
import os import
secrets

class SecurePasswordManager:     def __init__(self, iterations: int =
600000):         # High iteration count (PBKDF2-HMAC-SHA256 recommended
standard)          self.iterations = iterations         self.algorithm =
'sha256'

    def hash_password(self, password: str):          # Generate 128
bits (16 bytes) of OS-level secure entropy         salt =
secrets.token_bytes(16)

        # Derive key using HMAC-SHA256
hashed_password =
hashlib.pbkdf2_hmac(             self.algorithm,
password.encode('utf-8'),             salt,
self.iterations         )          return salt,
hashed_password

    def verify_password(self, password: str, salt: bytes, expected_hash: bytes) ->
bool:
        computed_hash =
hashlib.pbkdf2_hmac(             self.algorithm,
password.encode('utf-8'),             salt,
self.iterations         )          # secrets.compare_digest avoids
side-channel timing leaks          return
secrets.compare_digest(computed_hash, expected_hash)

# Demonstration Execution if
__name__ == "__main__":
    pm = SecurePasswordManager()
user_pass = "ComplexP@ssw0rd2026!"

    # Store salt and hash in the database     salt,
password_hash = pm.hash_password(user_pass)

    # Verification check     is_valid =
pm.verify_password(user_pass, salt, password_hash)
print(f"Password Verification Success: {is_valid}")
```

QUESTION 2: DIGITAL SIGNATURE FOR DOCUMENT AUTHENTICATION

1. Problem Statement & System Requirements

A company needs to ensure that official documents sent electronically across open networks are authentic and unaltered. In electronic document workflows, organizations face three primary risks:

- **Forgery / Impersonation:** An unauthorized third party generates a fraudulent document claiming to originate from an executive or authorized entity.
- **Tampering / Data Alteration:** An adversary intercepting the document modifies financial figures, contract terms, or legal clauses in transit.
- **Repudiation:** The legitimate sender transmits a binding document but later denies sending it to evade contractual obligations.

2. Part A: Mechanisms of Digital Signatures

Digital Signatures solve these vulnerabilities by combining **Asymmetric Public Key Cryptography** (e.g., RSA or ECDSA) with **Cryptographic Hash Functions** (e.g., SHA-256).

2.1. The Signing Phase (Sender Side)

1. **Message Digest Generation:** The document M is processed through a cryptographic hash function to compute a fixed-length message digest H(M) = SHA-256(M).
2. **Asymmetric Encryption:** The sender encrypts the hash digest H(M) using their private key d:

$$Signature = (H(M))^d \bmod n \text{ (RSA Signing)}$$

3. **Attachment & Transmission:** The resulting signature is attached to the electronic document M and transmitted to the recipient.

2.2. The Verification Phase (Recipient Side)

1. **Hash Recomputation:** The recipient receives (M, Signature) and independently calculates the hash digest of the received document H(M)' = SHA-256(M).
2. **Signature Decryption:** The recipient decrypts the signature using the sender's public key e:

$$H(M)'' = (Signature)^e \bmod n \text{ (RSA Verification)}$$

3. **Integrity & Authenticity Validation:** The recipient compares H(M)' against H(M)''. If H(M)' == H(M)'', the signature is validated.

3. Part B: Justification of Integrity, Authentication, and Non-Repudiation

3.1. Data Integrity Guarantee

Cryptographic hash functions exhibit the **Avalanche Effect**: changing even a single character, bit, or formatting space in document M causes an unpredictable, drastic shift in the output hash. If an attacker modifies the document during transit, the recipient's recomputed hash H(M)' will not match the decrypted hash H(M)'', immediately detecting any tampering.

3.2. Sender Authentication

In asymmetric cryptography, the private key d is mathematically linked to the public key e via prime factor relations (n = p * q), but deriving d from e is computationally infeasible due to the Integer Factorization Problem. Because only the legitimate owner possesses private key d, a signature successfully decrypted by public key e authenticates the sender's identity.

3.3. Non-Repudiation with Public Key Infrastructure (PKI)

Non-Repudiation prevents a sender from claiming that a signed document was forged by a third party. Because private key secret distribution is strictly restricted to the owner, no other entity can generate a signature that validates against the sender's public key.

To prevent a sender from claiming their public key was spoofed, a **Public Key Infrastructure (PKI)** uses trusted **Certificate Authorities (CAs)**. The CA issues a digital certificate binding the identity of the organization to their public key using X.509 standards.

4. Mathematical Workflow Comparison Table

Security Property	Cryptographic Mechanism	Failure Mode if Compromised
Authenticity	Private key signing by sender; Public key verification by receiver.	Attacker impersonates corporate sender if private key leaks.
Integrity	SHA-256 One-Way Hashing of full document content.	Unauthorized contract alterations go undetected.
Non-Repudiation	X.509 PKI Certificates binding identity to public keys.	Sender legally denies authorizing financial transactions.

5. Python Reference Implementation: RSA Digital Signature System

The following script provides a self-contained demonstration of digital signature generation, verification, and tamper detection using RSA key generation and SHA-256 hashing.

```
import hashlib

class RSADigitalSignature:
    def __init__(self):
        # Key generation parameters
        self.p = 61
        self.q = 53
        self.n = self.p * self.q # Modulus
        self.phi = (self.p - 1) * (self.q - 1)
        self.e = 17 # Public exponent
        self.d = self._mod_inverse(self.e, self.phi) # Private key

    def _extended_gcd(self, a, b):
        if a == 0:
            return b, 0, 1
        gcd, x1, y1 = self._extended_gcd(b % a, a)
        x = y1 - (b // a) * x1
        y = x1
        return gcd, x, y

    def _mod_inverse(self, e, phi):
        gcd, x, _ = self._extended_gcd(e, phi)
        if gcd != 1:
            raise ValueError("Modular inverse does not exist")
        return (x % phi + phi) % phi

    def _hash_document(self, document: str) -> int:
        digest = hashlib.sha256(document.encode('utf-8')).hexdigest()
        return int(digest, 16) % self.n

    def sign_document(self, document: str) -> int:
        h = self._hash_document(document)
        signature = pow(h, self.d, self.n)
        return signature

    def verify_signature(self, document: str, signature: int) -> bool:
        h_recomputed = self._hash_document(document)
        h_decrypted = pow(signature, self.e, self.n)
        return h_recomputed == h_decrypted

# Demonstration Execution
if __name__ == "__main__":
    dss = RSADigitalSignature()

    official_doc = "CONFIDENTIAL CONTRACT: Payment of $50,000 to Vendor A."
    tampered_doc = "CONFIDENTIAL CONTRACT: Payment of $90,000 to Vendor A."

    # 1. Sender signs the document
    signature = dss.sign_document(official_doc)
    print(f"Generated Digital Signature: {signature}")
```

```
    # 2. Recipient verifies valid document
    valid_check = dss.verify_signature(official_doc, signature)
    print(f"Verification of Authentic Document: {valid_check}")

    # 3. Recipient verifies tampered document
    tamper_check = dss.verify_signature(tampered_doc, signature)
    print(f"Verification of Tampered Document: {tamper_check}")
```

6. Summary & Recommendations

Modern web applications and digital enterprise systems must prioritize cryptographic defenses. Password management requires non-reversible, salted, and memory-hard key derivation algorithms like Argon2id or Bcrypt to withstand database leaks. For electronic document exchange, digital signatures backed by PKI and X.509 certificates provide a complete security framework that guarantees authenticity, integrity, and legal non-repudiation across public networks.